

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №4
по дисциплине «Методы тестирования программного
обеспечения»
«Автоматизированное тестирование»
Вариант 4

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А. М. _____
Курочкин М. А. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Постановка задачи	5
2 Описание средств автоматизации тестирования	6
2.1 JUnit	6
2.1.1 Особенности JUnit	6
2.1.2 Этапы работы с JUnit	6
2.2 Selenium WebDriver	7
2.2.1 Особенности Selenium WebDriver	7
2.2.2 Основные методы WebDriver	8
2.2.3 Этапы работы с WebDriver	9
2.3 TestNG	9
3 Задача 1. Юнит-тестирование библиотеки	10
3.1 Подготовительные операции	10
3.2 Тесты функции calculateCircleArea	10
3.3 Тесты функции calculateCirclePerimeter	11
3.4 Тесты функции calculateTriangleArea	12
3.5 Тесты функции calculateHypotenuse	12
3.6 Результаты	13
4 Задача 2. UI-тестирование веб-сайта	14
4.1 DriverSetup	15
4.1.1 Метод setup()	15
4.1.2 Метод exit()	16
4.2 Класс NavigationTrainingTest	16
4.2.1 Выполняемые проверки	16
4.2.2 Обработка ошибок	17
4.3 Класс HomePageNavigationTestsResult	17
4.3.1 Выполняемые проверки	17
4.3.2 Обработка ошибок	18
4.4 Результаты тестирования	18
Заключение	19
Список литературы	20
Приложение А. Исходный код GeometryTest	21
Приложение Б. Исходный код GeometryUtilsTest	22
Приложение В. Исходный код DriverSetup	24

Приложение Г. Исходный код NavigationTrainingResult	25
Приложение Д. Исходный код NavigationTrainingResult	28

Введение

Тестирование программного обеспечения — это процесс проверки соответствия реального поведения программы ожидаемым требованиям. Основная цель — выявить дефекты, убедиться в работоспособности ключевых функций и снизить риски сбоев в работе продукта.

Автоматизированное тестирование — это выполнение тестов с помощью специальных инструментов и фреймворков без постоянного участия человека. Автоматизация экономит время на повторяющихся проверках, повышает точность и позволяет быстрее получать обратную связь, но требует затрат на разработку и поддержку тестов.

1 Постановка задачи

Цель: Изучение и применение инструментов автоматизированного тестирования: фреймворка JUnit для модульного-тестирования и Selenium WebDriver вместе с TestNG для UI-тестирования веб-приложений.

Задачи:

1. Разработать unit-тесты для библиотеки lab-1.0.jar с использованием фреймворка JUnit.
 - (a) Разработать unit-тесты для четырех методов класса библиотеки lab-1.0.jar с использованием фреймворка JUnit.
 - (b) Запустить тесты через интегрированную среду разработки (IDE).
 - (c) Продемонстрировать результаты выполнения тестов.
2. Провести тестирование веб-сайта с использованием Selenium WebDriver и TestNG.
 - (a) Написать два теста, проверяющих корректное отображение и работу страниц веб-сайта.
 - (b) Организовать код тестов в соответствии с Java Code Convention.
 - (c) Запустить тесты через конфигурационный файл TestNG suite xml.

2 Описание средств автоматизации тестирования

2.1 JUnit

JUnit — это фреймворк для написания и выполнения модульных тестов на языке программирования Java. Он позволяет разработчикам создавать и повторять тесты для проверки правильности работы кода.

2.1.1 Особенности JUnit

- **Аннотации.** JUnit предоставляет различные аннотации для задания и настройки тестов.
- **Поддержка параметризованных тестов.** С помощью аннотаций `@Parameterized` и `@ValueSource` можно передавать параметры в тесты.
- **Интеграция с различными средами разработки.** JUnit легко интегрируется с большинством IDE, таких как IntelliJ IDEA, Eclipse и NetBeans.
- **Простота использования.** JUnit легко освоить, что делает его популярным среди программистов.

2.1.2 Этапы работы с JUnit

1. **Создание тестов.** Написание тестового метода и добавление к нему аннотации `@Test`.
2. **Настройка тестов.** Настройка происходит при помощи аннотаций, предоставляемых JUnit:
 - **@BeforeEach:** Указывает, что метод должен выполняться перед каждым тестовым методом в классе. Используется для подготовки общего окружения (например, инициализации объектов или сброса данных) перед запуском каждого отдельного теста.
 - **@AfterEach:** Помечает метод, который запускается после каждого тестового метода. Применяется для очистки ресурсов или сброса состояния, чтобы избежать влияния одного теста на другой.
 - **@BeforeAll:** Обозначает статический метод, выполняющийся один раз до запуска всех тестов в классе. Используется для ресурсоёмкой инициализации, например, подключения к базе данных или запуска сервера.
 - **@AfterAll:** Указывает на статический метод, который запускается один раз после завершения всех тестов класса. Нужен для финального

освобождения глобальных ресурсов, таких как закрытие соединений или остановка сервера.

- **@DisplayName:** Позволяет задать произвольное человеко-читаемое имя для тестового класса или метода. Используется для улучшения читаемости отчётов и логирования вместо имени метода в коде.
- **@Disable:** Отключает выполнение помеченного тестового метода или целого класса. Применяется для временного пропуска тестов, например, если функциональность еще не реализована или нестабильна.
- **@Nested:** Позволяет создавать вложенные классы с тестами внутри основного тестового класса. Используется для логической группировки тестов и создания иерархии с общим контекстом.
- **@Tag:** Назначает метки для фильтрации тестов на уровне метода или класса. Используется в сборках для выборочного запуска только определённых категорий тестов
- **@TestFactory:** Обозначает метод, который динамически генерирует тесты, возвращая коллекцию `DynamicTest`. Применяется, когда набор тестов неизвестен на этапе компиляции, например, для проверки множества внешних входных данных.

3. **Запуск тестов.** Выполнение тестов с помощью встроенных инструментов IDE или командной строки.

2.2 Selenium WebDriver

Selenium WebDriver — это библиотека и программный интерфейс для автоматизации действий веб-браузера. WebDriver управляет браузером напрямую через нативный для каждого браузера драйвер (например, `ChromeDriver` для Google Chrome), что позволяет точно имитировать реальные действия пользователя: клики, ввод текста, навигацию, работу с выпадающими списками или вкладками. WebDriver поддерживает множество языков (Java, Python, C#, JavaScript и др.) и предоставляет гибкие механизмы поиска элементов (по ID, CSS, XPath), ожидания загрузки страницы и выполнения JavaScript.

2.2.1 Особенности Selenium WebDriver

1. **Прямое управление браузером через драйвер.** Взаимодействует с браузером нативно, полностью имитируя действия реального пользователя (клики, ввод, скролл).
2. **Кросс-браузерность.** Один и тот же код можно запускать в Chrome, Firefox, Edge, Safari без изменений.

3. **Синхронизация (ожидания).** Встроенные механизмы явных (`WebDriverWait`) и неявных (`implicitly_wait`) ожиданий для работы с динамическими страницами.
4. **Многоязычность.** Поддерживает Java, Python, C#, JavaScript, Ruby и другие языки.

2.2.2 Основные методы WebDriver

- **Навигация:**

- `get(String url)` – открывает указанный URL в текущем окне браузера,
- `navigate().back()` – эмулирует нажатие кнопки «Назад» в браузере,
- `navigate().forward()` – эмулирует нажатие кнопки «Вперед»,
- `navigate().refresh()` – перезагружает текущую страницу;

- **Управление окном:**

- `manage().window().maximize()` – разворачивает окно на весь экран,
- `getWindowHandle()` / `getWindowHandles()` – получение идентификаторов окон,
- `switchTo().window(String handle)` – переключение между вкладками/окнами;

- **Поиск элементов:**

- `findElement(By locator)` – поиск одного элемента,
- `findElements(By locator)` – поиск всех элементов по локатору,
- `By.id(String id)` / `By.className(String name)` / `By.xpath(String xpath)` / `By.cssSelector(String selector)` – основные локаторы;

- **Методы интерфейса WebElement:**

- `click()` – клик по элементу,
- `sendKeys(CharSequence... keys)` – ввод текста,
- `clear()` – очистка поля,
- `getText()` – получение текста элемента,
- `isDisplayed()` / `isEnabled()` / `isSelected()` – проверка состояния элемента;

- **Дополнительные возможности:**

- `executeScript(String script, Object... args)` – выполнение JavaScript,

- `manage().timeouts().implicitlyWait(Duration d)` – неявное ожидание,
- `WebDriverWait с ExpectedConditions` – явные ожидания,
- `switchTo().alert()` – работа с всплывающими окнами,
- `switchTo().frame()` – переключение в `iframe`,
- `getTitle()` / `getCurrentUrl()` – получение заголовка и URL страницы,
- `quit()` – завершение сессии и закрытие браузера.

2.2.3 Этапы работы с WebDriver

1. **Инициализация драйвера.** Создание экземпляра объекта `WebDriver` для конкретного браузера. В этот момент запускается браузер, и `WebDriver` устанавливает с ним соединение.
2. **Переход на нужную страницу.** Вызов метода `get(String url)`, который открывает указанный URL. После этого можно получить заголовок страницы, текущий URL или исходный код для проверок.
3. **Поиск элементов на странице.** Использование `findElement(By.locator)` с различными локаторами для получения ссылки на веб-элемент.
4. **Взаимодействие с элементами.** Выполнение действий над найденными элементами: `click()`, `sendKeys()`, `clear()`, `getText()`.
5. **Проверка ожидаемого результата.** Сравнение фактического состояния страницы с ожидаемым с помощью библиотек, например, `JUnit` или `TestNG`.
6. **Завершение работы.** Вызов `quit()` для закрытия всех окон браузера и освобождения ресурсов (процесс драйвера завершается).

2.3 TestNG

`TestNG` — это фреймворк для автоматизированного тестирования программного обеспечения, написанный на языке `Java`. Он также как и `JUnit` предназначен для создания и запуска тестов для проверки функциональности и качества программных продуктов. В отличие от `JUnit`, работает для функционального и более высоких уровней тестирования.

`TestNG` также поддерживает параллельное выполнение тестов, что позволяет ускорить процесс тестирования и повысить производительность. Кроме того, фреймворк интегрируется с другими инструментами и системами, такими как `Jenkins`, `Maven` и `Eclipse`, что упрощает процесс тестирования и развертывания программного обеспечения.

3 Задача 1. Юнит-тестирование библиотеки

Дано: Библиотечный файл `lab-1.0.jar`, класс `GeometryUtils`.

Требуется: Разработать проект автоматизированного модульного-тестирования для предоставленной библиотеки.

Задачи:

1. Создать java проект, подключить библиотеку `lab-1.0.jar` и зависимость `JUnit`.
2. Выбрать четыре метода из предоставленной библиотеки по своему усмотрению и написать для них юнит-тесты с использованием фреймворка `JUnit`.
3. Запустить тесты через интегрированную среду разработки (IDE).
4. Продемонстрировать результаты выполнения.

Выбранные методы:

- `calculateCircleArea` – расчет площади круга,
- `calculateCirclePerimeter` – расчет длины окружности,
- `calculateTriangleArea` – расчет периметра треугольника,
- `calculateHypotenuse` – расчет длины гипотенузы.

3.1 Подготовительные операции

Перед разработкой тестов был создан тестовый класс `GeometryUtilsTest`, предназначенный для проверки методов класса `GeometryUtils`. Поскольку все методы тестируемого класса являются статическими, создание экземпляра класса не требуется. В тестовом классе используется аннотация `@BeforeAll` для выполнения инициализационных действий перед запуском всех тестов.

Для корректного сравнения значений типа `double` введена константа `DELTA = 1e-6`.

3.2 Тесты функции `calculateCircleArea`

Функция `calculateCircleArea(double radius)` вычисляет площадь круга по формуле: $S = \pi r^2$. На вход функция принимает параметр `radius` типа `double`, а возвращает площадь. При $radius \leq 0$ выбрасывается `IllegalArgumentException`.

Исходя из спецификации, были составлены тесты с использованием техник классов эквивалентности и анализа граничных значений. Классы эквивалентности представлены в табл. 1.

Таблица 1. Классы эквивалентности

Входное условие	Допустимые классы	Недопустимые классы
radius	$\text{radius} > 0$	$\text{radius} \leq 0$

Граничные значения:

- $\text{radius} = \text{Double.MIN_VALUE}$
- $\text{radius} = 0$
- $\text{radius} = -1$
- $\text{radius} = \text{Double.MAX_VALUE}$

Результаты тестирования приведены в табл. 2.

Таблица 2. Результаты тестов

radius	Ожидаемый результат	Фактический результат	Статус теста
1.0	π	π	Пройден
MIN_VALUE	Площадь круга	Площадь круга	Пройден
MAX_VALUE	Площадь круга	Площадь круга	Пройден
0.0	Exception IllegalArgumentException	Exception IllegalArgumentException	Пройден
-1.0	Exception IllegalArgumentException	Exception IllegalArgumentException	Пройден

Все тесты `calculateCircleArea` проходит успешно.

3.3 Тесты функции `calculateCirclePerimeter`

Функция `calculateCirclePerimeter` должна вычислять длину окружности по формуле $C = 2\pi r$, однако в исходном коде допущена ошибка: вместо $2\pi r$ возвращается πr .

Классы эквивалентности и граничные значения аналогичны предыдущему методу. Классы эквивалентности представлены в табл. 3.

Таблица 3. Классы эквивалентности

Входное условие	Допустимые классы	Недопустимые классы
radius	$\text{radius} > 0$	$\text{radius} \leq 0$

Результаты тестирования приведены в табл. 4.

Таблица 4. Результаты тестов

radius	Ожидаемый результат	Фактический результат	Статус теста
1.0	2π	π	Не пройден
MAX_VALUE	$2\pi \cdot \text{MAX_VALUE}$	$\pi \cdot \text{MAX_VALUE}$	Не пройден
-1.0	Exception IllegalArgumentException	Exception IllegalArgumentException	Пройден

Тесты для положительных значений радиуса не проходят, так как результат отличается от ожидаемого в два раза.

3.4 Тесты функции calculateTriangleArea

Функция `calculateTriangleArea` вычисляет площадь треугольника по формуле $S = \frac{1}{2}bh$. Она принимает на вход два параметра: `base` и `height`.

Классы эквивалентности представлены в табл. 5.

Таблица 5. Классы эквивалентности

Входное условие	Допустимые классы	Недопустимые классы
base, height	base > 0 и height > 0	base ≤ 0 или height ≤ 0

Граничные значения:

- base = MIN_VALUE
- height = MIN_VALUE
- base = 0
- height = 0
- $base \leq 0, height \leq 0$

Результаты тестирования приведены в табл. 6.

Таблица 6. Результаты тестов

base, height	Ожидаемый результат	Фактический результат	Статус теста
2, 4	4	4	Пройден
MIN_VALUE, MIN_VALUE	Площадь треугольника	Площадь треугольника	Пройден
0, 5	Exception IllegalArgumentException	Exception IllegalArgumentException	Пройден
-1, 3	Exception IllegalArgumentException	Exception IllegalArgumentException	Пройден

Все тесты `calculateCircleArea` проходит успешно.

3.5 Тесты функции calculateHypotenuse

Функция `calculateHypotenuse` вычисляет длину гипотенузы треугольника по формуле $c = \sqrt{a^2 + b^2}$. Она принимает на вход два параметра: `a` и `b`.

Классы эквивалентности представлены в табл. 7.

Таблица 7. Классы эквивалентности

Входное условие	Допустимые классы	Недопустимые классы
a, b	a > 0 и b > 0	a ≤ 0 или b ≤ 0

Результаты тестирования приведены в табл. 8.

Таблица 8. Результаты тестов

a, b	Ожидаемый результат	Фактический результат	Статус теста
3, 4	5	5	Пройден
1, 1	$\sqrt{2}$	$\sqrt{2}$	Пройден
0, 5	Exception IllegalArgumentException	Exception IllegalArgumentException	Пройден
-1, 2	Exception IllegalArgumentException	Exception IllegalArgumentException	Пройден

Все тесты `calculateCircleArea` проходит успешно.

3.6 Результаты

Разработано 16 тестов. Из них:

- `calculateCircleArea` – все пройдены;
- `calculateCirclePerimeter` – тесты для положительных радиусов не пройдены;
- `calculateTriangleArea` – все пройдены;
- `calculateHypotenuse` – все пройдены.

```

1 GeometryUtilsTest > testTriangleAreaValid_1 () PASSED
2 GeometryUtilsTest > testTriangleAreaValid_2 () PASSED
3
4 GeometryUtilsTest > testCircleAreaValid_1 () PASSED
5 GeometryUtilsTest > testCircleAreaValid_2 () PASSED
6 GeometryUtilsTest > testCircleAreaValid_3 () PASSED
7
8 GeometryUtilsTest > testHypotenuseInvalid_1 () PASSED
9 GeometryUtilsTest > testHypotenuseInvalid_2 () PASSED
10
11 GeometryUtilsTest > testCirclePerimeterInvalid_1 () PASSED
12
13 GeometryUtilsTest > testTriangleAreaInvalid_1 () PASSED
14 GeometryUtilsTest > testTriangleAreaInvalid_2 () PASSED
15
16 GeometryUtilsTest > testCirclePerimeterValid_1 () FAILED org.opentest4j.
    AssertionError: expected: 6.283185307179586 but was:
    3.141592653589793
17 GeometryUtilsTest > testCirclePerimeterValid_2 () FAILED org.opentest4j.
    AssertionError: expected: 12.566370614359172 but was:
    6.283185307179586
18
19 GeometryUtilsTest > testHypotenuseValid_1 () PASSED
20 GeometryUtilsTest > testHypotenuseValid_2 () PASSED
21
22 GeometryUtilsTest > testCircleAreaInvalid_1 () PASSED
23 GeometryUtilsTest > testCircleAreaInvalid_2 () PASSED
24
25 16 tests completed, 2 failed

```

4 Задача 2. UI-тестирование веб-сайта

Дано:

- Веб-сайт для тестирования: <https://jdi-testing.github.io/jdi-light/index.html>
- Драйвер для работы с браузером.
- Фреймворк: Selenium, TestNG.

Требуется:

- Подключить в проекте зависимости Selenium и TestNG, а также добавить драйвер для работы с браузером в проект.
- Создать два теста на языке Java с использованием Selenium WebDriver и фреймворка TestNG для проверки корректности отображения страниц сайта <https://jdi-testing.github.io/jdi-light/index.html> в браузере.
- Тесты должны соответствовать стандартам Java Code Convention и запускаться с помощью TestNG suite xml.
- Первый тест должен проверять основные элементы главной страницы (иконки преимуществ, тексты, iframe, левое меню).
- Второй тест должен проверять страницу с различными элементами (кнопки, чекбоксы, радиокнопки, выпадающий список).

Первый и второй тест должны проигрывать 1 и 2 тестовый сценарий соответственно. Тестовые сценарии представлены в табл. 9-13.

Таблица 9. Тестовый сценарий 1

№	Шаг	Данные	Ожидаемый результат
1	Перейти на страницу "Different elements" через главное меню	Пункт меню: "Service" → "Different elements"	Страница Different Elements открыта
2	Проверить, что заголовок страницы содержит слова "different" и "element"	Ожидаемый текст: "different" и "element" (без учёта регистра)	Заголовок содержит оба слова
3	Проверить наличие не менее 4 кнопок	Поиск по селектору: 'input[type='button']', 'button'	Найдено минимум 4 кнопки
4	Проверить наличие не менее 4 чекбоксов	Поиск по селектору: 'input[type='checkbox']'	Найдено минимум 4 чекбокса
5	Проверить наличие не менее 4 радиокнопок	Поиск по селектору: 'input[type='radio']'	Найдено минимум 4 радиокнопки
6	Проверить наличие хотя бы одного выпадающего списка	Поиск по тегу: 'select'	Найден хотя бы 1 select элемент
7	Проверить, что выпадающий список отображается и содержит не менее 2 элементов	Проверка видимости элемента + поиск по тегу 'option' внутри select	Выпадающий список видим и содержит минимум 2 опции
8	Проверить наличие значений "Red" и "Green" в списке	Ожидаемые значения: "Red" "Green"	Оба цвета присутствуют в списке опций
9	Проверить наличие подписей чекбоксов (label-checkbox)	Поиск по классу: 'label-checkbox'	Найдено минимум 2 лейбла

Таблица 10. Тест 1. Заголовок страницы

№	Шаг	Данные	Ожидаемый результат
1	Получить заголовок страницы	<code>driver.getTitle()</code>	Заголовок получен
2	Проверить, что заголовок не пустой	Текст заголовка	Заголовок не пустой
3	Проверить наличие слова "home"	"home" (без учёта регистра)	Заголовок содержит "home"
4	Проверить точное соответствие	"Home Page"	Заголовок равен "Home Page"

Таблица 11. Тест 2. Верхнее меню

№	Шаг	Данные	Ожидаемый результат
1	Найти пункты меню	<code>ul.uii-navigation.nav > li</code>	Найдено 4 элемента
2	Проверить тексты пунктов	HOME, CONTACT FORM, SERVICE, METALS	Тексты совпадают
3	Проверить ссылки	href атрибут	Ссылки корректны
4	Проверить кликабельность	<code>isDisplayed, isEnabled</code>	Все элементы активны

Таблица 12. Тест 3. Пользователь

№	Шаг	Данные	Ожидаемый результат
1	Найти имя пользователя	<code>id="user-name"</code>	Элемент найден
2	Проверить отображение	<code>isDisplayed()</code>	Элемент видим
3	Проверить текст	"ROMAN "IOVLEV"	Имя содержит ожидаемые значения
4	Проверить формат	regex	Только буквы и пробелы

Таблица 13. Тест 4. UI элементы страницы

№	Шаг	Данные	Ожидаемый результат
1	Проверка аватара	<code>.profile-photo</code>	Элемент отображается
2	Проверка логотипа	<code>.logo / img</code>	Логотип найден
3	Проверка футера	<code>footer</code>	Футер отображается
4	Проверка контейнера	<code>.container</code>	Контейнер найден

4.1 DriverSetup

Первоначально требовалось настроить класс `DriverSetup`. Он предназначен для инициализации и настройки `WebDriver` в тестовом проекте.

4.1.1 Метод `setup()`

Метод с аннотацией `@BeforeTest` выполняется перед всеми тестами и выполняет следующие действия:

1. Создает экземпляр драйвера для браузера Google Chrome;
2. Переходит по URL тестового стенда с помощью метода `navigate().to()`;
3. Выполняет процедуру авторизации: клик по иконке пользователя, ввод логина и пароля, нажатие кнопки входа.

4.1.2 Метод `exit()`

Метод с аннотацией `@AfterTest` закрывает браузер после выполнения всех тестов, освобождая ресурсы.

4.2 Класс `NavigationTrainingTest`

Класс `NavigationTrainingTest` наследуется от `DriverSetup` и содержит автоматизированные тесты для проверки элементов главной страницы. Все тесты используют обычные утверждения `TestNG` и аннотацию `@Test`.

4.2.1 Выполняемые проверки

1. Проверка перехода на страницу. Выполняется переход на страницу “Different elements” через главное меню навигации (пункт “Service”).
2. Проверка заголовка страницы. Полученный заголовок страницы приводится к нижнему регистру и проверяется на наличие подстрок “different” и “element”. При отсутствии хотя бы одной подстроки тест фиксирует ошибку с указанием фактического значения заголовка.
3. Проверка наличия кнопок. Выполняется поиск элементов `<button>` и `input[type='button']`. Проверяется, что количество найденных элементов не менее четырех. В случае меньшего количества фиксируется ошибка с указанием фактического числа элементов.
4. Проверка наличия чекбоксов. Выполняется поиск элементов `input[type='checkbox']`. Проверяется, что количество найденных элементов не менее четырех.
5. Проверка наличия радиокнопок. Выполняется поиск элементов `input[type='radio']`. Проверяется, что количество найденных элементов не менее четырех.
6. Проверка наличия выпадающего списка. Выполняется поиск элементов `<select>`. Проверяется, что на странице присутствует хотя бы один такой элемент.
7. Проверка видимости и содержимого выпадающего списка. Проверяется, что найденный элемент `<select>` отображается на странице (метод `isDisplayed`). Затем выполняется поиск элементов `<option>` внутри списка и проверяется, что их количество не менее двух.
8. Проверка содержимого выпадающего списка. Выполняется проверка наличия значений “Red” и “Green” среди элементов `<option>`.
9. Проверка наличия подписей чекбоксов. Выполняется поиск элементов с классом `label-checkbox`. Проверяется, что количество таких элементов не менее двух.

4.2.2 Обработка ошибок

При возникновении ошибки в ходе выполнения проверки тест не прерывается, так как используется механизм мягких проверок (SoftAssert). Все обнаруженные ошибки накапливаются и фиксируются при вызове метода `assertAll()`. В случае возникновения исключений дополнительно сохраняется отладочная информация (скриншот и HTML-код страницы), что позволяет упростить анализ причин ошибки.

4.3 Класс `HomePageNavigationTestsResult`

Класс `HomePageNavigationTestsResult` наследуется от `DriverSetup` и предназначен для тестирования главной страницы веб-приложения. Каждый тест реализован как отдельный метод с аннотацией `@Test`. Для управления порядком выполнения используется параметр `priority`.

4.3.1 Выполняемые проверки

1. Проверка заголовка страницы. Полученный заголовок страницы проверяется на непустое значение, наличие подстроки “home” (без учета регистра), а также на точное соответствие строке “Home Page”.
2. Проверка количества пунктов верхнего меню. Выполняется поиск элементов меню по CSS-селектору. Проверяется, что количество найденных элементов равно четырем.
3. Проверка текстов пунктов меню. Тексты элементов меню приводятся к верхнему регистру и сравниваются с ожидаемыми значениями.
4. Проверка ссылок меню. Проверяется наличие атрибута `href` у ссылок, их отображение и активность. Дополнительно проверяется соответствие значения `href` ожидаемым шаблонам.
5. Проверка кликабельности пунктов меню. Для каждого элемента проверяется, что он отображается и доступен для взаимодействия (методы `isDisplayed`, `isEnabled`).
6. Проверка имени пользователя. Проверяется наличие элемента, его видимость, непустое значение текста, соответствие ожидаемым значениям и корректность формата (регулярное выражение).
7. Проверка иконки пользователя. Проверяется наличие элемента, его видимость, активность и ненулевые размеры. Дополнительно проверяется наличие визуального содержимого.

8. Проверка логотипа сайта. Проверяется наличие логотипа, его видимость, наличие атрибута alt, а также корректность ссылки (переход на главную страницу).
9. Проверка футера страницы. Проверяется наличие футера, его видимость и непустое текстовое содержимое. Дополнительно проверяется наличие ожидаемых ключевых элементов (копирайт, год и т.д.).
10. Проверка адаптивных классов. Проверяется наличие основного контейнера страницы и наличие у него классов, отвечающих за адаптивную верстку.

4.3.2 Обработка ошибок

При возникновении ошибки в ходе выполнения тестов используется механизм стандартных проверок TestNG (Assert), при котором выполнение текущего теста прерывается при первом несоответствии ожидаемому результату. Это позволяет быстро локализовать причину ошибки. В случае падения теста в отчёте выводится сообщение с описанием несоответствия и фактическим значением, что упрощает анализ и отладку.

4.4 Результаты тестирования

С помощью тестов были проверены основные элементы главной страницы, а также такие элементы, как кнопки, чекбоксы, радиокнопки, выпадающие списки.

```
1 NavigationTrainingResult > testDifferentElementsPageFullCheck () PASSED
2
3 HomePageNavigationTestsResult > testPageTitle () PASSED
4 HomePageNavigationTestsResult > testHeaderMenuCount () PASSED
5 HomePageNavigationTestsResult > testHeaderMenuTexts () PASSED
6 HomePageNavigationTestsResult > testHeaderMenuLinks () PASSED
7 HomePageNavigationTestsResult > testHeaderMenuClickable () PASSED
8 HomePageNavigationTestsResult > testLoggedInUserName () PASSED
9 HomePageNavigationTestsResult > testUserAvatar () PASSED
10 HomePageNavigationTestsResult > testSiteLogo () PASSED
11 HomePageNavigationTestsResult > testFooterContent () PASSED
12 HomePageNavigationTestsResult > testResponsiveClasses () PASSED
13
14 11 tests completed, 0 failed
```

Заключение

В ходе выполнения лабораторной работы были изучены основные подходы к автоматизированному тестированию программного обеспечения. Были получены практические навыки разработки и выполнения модульных тестов с использованием фреймворка JUnit, а также применения техник классов эквивалентности и анализа граничных значений при проектировании тестов. В результате тестирования были выявлены случаи некорректной работы функций, что подтвердило эффективность использования автоматизированных тестов для поиска ошибок.

Кроме того, были приобретены навыки тестирования веб-приложений с использованием инструментов Selenium WebDriver и TestNG. Были реализованы тестовые сценарии для проверки пользовательского интерфейса, навигации и содержимого веб-страниц, а также освоены методы организации тестов, обработки ошибок и формирования отчётности.

Результаты работы показали, что автоматизированное тестирование позволяет существенно повысить скорость и повторяемость проверок, однако не исключает необходимость ручного тестирования, которое помогает выявить нестандартные и трудноформализуемые проблемы. Также было установлено, что сами автоматические тесты требуют проверки и поддержки, поскольку они являются частью программного кода.

Полученные знания и навыки, а также использование фреймворков JUnit, Selenium и TestNG, могут быть применены в дальнейшем при разработке и тестировании программных систем, способствуя повышению качества и надёжности программного обеспечения.

Список литературы

1. Г. Майерс, Т. Баджетт, К. Сандлер. Искусство тестирования программ 3-е издание. Санкт-Петербург: «Диалектика», 2012. 295 с.
2. Selenium Documentation. – URL:<https://www.selenium.dev/documentation/>, (дата обращения 26.04.2026)
3. TestNG Documentation. – URL: <https://testng.org/doc/>, (дата обращения 26.04.2026)
4. JUnit 5 User Guide. – URL: <https://junit.org/junit5/docs/current/userguide/>, (дата обращения 26.04.2026)

Приложение А. Исходный код GeometryTest

```
1 import org.junit.jupiter.api.AfterAll;
2 import org.junit.jupiter.api.BeforeAll;
3
4 public class GeometryTest {
5
6     protected static final double DELTA = 1e-6;
7
8     @BeforeAll
9     static void setUp() {
10         System.out.println("=== Start tests ===");
11     }
12
13     @AfterAll
14     static void tearDown() {
15         System.out.println("=== End tests ===");
16     }
17 }
```

Приложение Б. Исходный код GeometryUtilsTest

```
1 import org.junit.jupiter.params.ParameterizedTest;
2 import org.junit.jupiter.params.provider.ValueSource;
3 import org.junit.jupiter.params.provider.CsvSource;
4 import ru.spbstu.hsai.GeometryUtils;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 class GeometryUtilsTest extends GeometryTest {
9
10     @ParameterizedTest
11     @ValueSource(doubles = {
12         1.0,
13         2.5,
14         Double.MIN_VALUE,
15         Double.MAX_VALUE
16     })
17     void testCircleAreaValid(double radius) {
18         double expected = Math.PI * radius * radius;
19         double actual = GeometryUtils.calculateCircleArea(radius);
20         assertEquals(expected, actual, DELTA);
21     }
22
23     @ParameterizedTest
24     @ValueSource(doubles = {
25         0.0,
26         -0.0,
27         -1.0,
28         -Double.MAX_VALUE
29     })
30     void testCircleAreaInvalid(double radius) {
31         assertThrows(IllegalArgumentException.class,
32             () -> GeometryUtils.calculateCircleArea(radius));
33     }
34
35
36     @ParameterizedTest
37     @ValueSource(doubles = {
38         1.0,
39         2.0,
40         Double.MAX_VALUE
41     })
42     void testCirclePerimeterValid(double radius) {
43         double expected = 2 * Math.PI * radius;
44         double actual = GeometryUtils.calculateCirclePerimeter(radius);
45         assertEquals(expected, actual, DELTA); // ДОЛЖЕН ПАДАТЬ
46     }
47
48     @ParameterizedTest
49     @ValueSource(doubles = {
50         0.0,
51         -1.0,
52         -Double.MAX_VALUE
53     })
54     void testCirclePerimeterInvalid(double radius) {
55         assertThrows(IllegalArgumentException.class,
56             () -> GeometryUtils.calculateCirclePerimeter(radius));
57     }
58 }
```

```

59
60 @ParameterizedTest
61 @CsvSource({
62     "2, 4, 4",
63     "3, 6, 9",
64     "1.5, 2, 1.5",
65     "1E-10, 1E-10, 5E-21"
66 })
67 void testTriangleAreaValid(double base, double height, double expected)
68 {
69     double actual = GeometryUtils.calculateTriangleArea(base, height);
70     assertEquals(expected, actual, DELTA);
71 }
72
73 @ParameterizedTest
74 @CsvSource({
75     "0, 5",
76     "-1, 3",
77     "2, -4",
78     "-1, -1"
79 })
80 void testTriangleAreaInvalid(double base, double height) {
81     assertThrows(IllegalArgumentException.class,
82         () -> GeometryUtils.calculateTriangleArea(base, height));
83 }
84
85 @ParameterizedTest
86 @CsvSource({
87     "3, 4, 5",
88     "1, 1, 1.414213562",
89     "5, 12, 13",
90     "1E-10, 1E-10, 1.414213562E-10"
91 })
92 void testHypotenuseValid(double a, double b, double expected) {
93     double actual = GeometryUtils.calculateHypotenuse(a, b);
94     assertEquals(expected, actual, DELTA);
95 }
96
97 @ParameterizedTest
98 @CsvSource({
99     "0, 5",
100     "-1, 2",
101     "3, -4",
102     "-1, -1"
103 })
104 void testHypotenuseInvalid(double a, double b) {
105     assertThrows(IllegalArgumentException.class,
106         () -> GeometryUtils.calculateHypotenuse(a, b));
107 }
108 }

```

Приложение В. Исходный код DriverSetup

```
1 package ru.spbstu.hsai;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.WebDriver;
5 import org.openqa.selenium.chrome.ChromeDriver;
6 import org.testng.annotations.AfterTest;
7 import org.testng.annotations.BeforeTest;
8
9 public class DriverSetup {
10     protected static WebDriver driver;
11
12     @BeforeTest
13     public static void setup() {
14         driver = new ChromeDriver();
15
16         driver.navigate().to("https://jdi-testing.github.io/jdi-light/index.html");
17
18         driver.findElement(By.cssSelector("html > body > header > div > nav > ul.ui-navigation.navbar-nav.navbar-right > li > a > span")).click();
19
20         driver.findElement(By.id("name")).sendKeys("Roman");
21         driver.findElement(By.id("password")).sendKeys("Jdi1234");
22         driver.findElement(By.id("login-button")).click();
23     }
24
25     @AfterTest
26     public static void exit() {
27         //10. Close Browser
28         driver.close();
29     }
30 }
```

Приложение Г. Исходный код NavigationTrainingResult

```
1 package ru.spbstu.hsai;
2
3 import org.openqa.selenium.*;
4 import org.openqa.selenium.interactions.Actions;
5 import org.openqa.selenium.support.ui.ExpectedConditions;
6 import org.openqa.selenium.support.ui.WebDriverWait;
7 import org.testng.annotations.Test;
8 import org.testng.asserts.SoftAssert;
9
10 import java.io.File;
11 import java.io.FileWriter;
12 import java.io.IOException;
13 import java.time.Duration;
14 import java.util.List;
15
16 public class NavigationTrainingResult extends DriverSetup {
17
18     protected void navigateTo(String menuItem) {
19         WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds
20             (15));
21         Actions actions = new Actions(driver);
22
23         try {
24             WebElement serviceBtn = wait.until(
25                 ExpectedConditions.elementToBeClickable(
26                     By.xpath("//span[text()='Service']")
27                 )
28             );
29             actions.moveToElement(serviceBtn).perform();
30             Thread.sleep(800);
31
32             WebElement targetBtn = wait.until(
33                 ExpectedConditions.elementToBeClickable(
34                     By.xpath("//span[text()='\" + menuItem + "\']")
35                 )
36             );
37
38             ((JavascriptExecutor) driver).executeScript(
39                 "arguments[0].scrollIntoView(true);", targetBtn
40             );
41
42             Thread.sleep(500);
43             targetBtn.click();
44             Thread.sleep(1500);
45
46         } catch (Exception e) {
47             throw new RuntimeException("Ошибка перехода: " + menuItem, e);
48         }
49     }
50
51     private void saveDebugInfo(String testName, Exception e) {
52         try {
53             File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType
54                 .FILE);
55             src.renameTo(new File("error_" + testName + ".png"));
56
57             FileWriter writer = new FileWriter("error_" + testName + ".html
```

```

57         ");
58         writer.write(driver.getPageSource());
59         writer.close();
60     } catch (IOException ex) {
61         ex.printStackTrace();
62     }
63 }
64
65 @Test
66 public void testDifferentElementsPageFullCheck() {
67
68     SoftAssert softAssert = new SoftAssert();
69     String testName = "DifferentElements";
70
71     try {
72         // 1. Переход
73         navigateTo("Different elements");
74
75         // 2. Title
76         String title = driver.getTitle().toLowerCase();
77         softAssert.assertTrue(
78             title.contains("different") && title.contains("element")
79             ,
80             "Неверный заголовок: " + title
81         );
82
83         // 3. Кнопки
84         List<WebElement> buttons =
85             driver.findElements(By.cssSelector("input[type='button
86             '], button"));
87         softAssert.assertTrue(buttons.size() >= 4,
88             "Меньше 4 кнопок: " + buttons.size());
89
90         // 4. Checkbox
91         List<WebElement> checkboxes =
92             driver.findElements(By.cssSelector("input[type='checkbox
93             ']"));
94         softAssert.assertTrue(checkboxes.size() >= 4,
95             "Меньше 4 чекбоксов: " + checkboxes.size());
96
97         // 5. Radio
98         List<WebElement> radios =
99             driver.findElements(By.cssSelector("input[type='radio
100             ']"));
101         softAssert.assertTrue(radios.size() >= 4,
102             "Меньше 4 радио: " + radios.size());
103
104         // 6. Dropdown
105         List<WebElement> selects = driver.findElements(By.tagName("
106             select"));
107         softAssert.assertTrue(selects.size() >= 1,
108             "Dropdown не найден");
109
110         if (!selects.isEmpty()) {
111             WebElement dropdown = selects.get(0);
112
113             // 7. Видимость + опции
114             softAssert.assertTrue(dropdown.isDisplayed(),
115                 "Dropdown скрыт");
116         }
117     }
118 }

```

```

111
112         List<WebElement> options = dropdown.findElements(By.tagName
113             ("option"));
114         softAssert.assertTrue(options.size() >= 2,
115             "Мало опций: " + options.size());
116
117         // 8. Проверка значений
118         boolean hasRed = false;
119         boolean hasGreen = false;
120
121         for (WebElement opt : options) {
122             String text = opt.getText().toLowerCase();
123             if (text.contains("red")) hasRed = true;
124             if (text.contains("green")) hasGreen = true;
125         }
126
127         softAssert.assertTrue(hasRed && hasGreen,
128             "Нет Red или Green");
129     }
130
131     // 9. Labels
132     List<WebElement> labels =
133         driver.findElements(By.className("label-checkbox"));
134     softAssert.assertTrue(labels.size() >= 2,
135         "Мало label-checkbox: " + labels.size());
136
137     } catch (Exception e) {
138         saveDebugInfo(testName, e);
139         softAssert.fail("Ошибка в тесте: " + e.getMessage());
140     }
141
142     softAssert.assertAll();
143 }

```

Приложение Д. Исходный код NavigationTrainingResult

```
1 package ru.spbstu.hsai;
2
3 import org.openqa.selenium.*;
4 import org.openqa.selenium.interactions.Actions;
5 import org.openqa.selenium.support.ui.ExpectedConditions;
6 import org.openqa.selenium.support.ui.WebDriverWait;
7 import org.testng.annotations.Test;
8 import org.testng.asserts.SoftAssert;
9
10 import java.io.File;
11 import java.io.FileWriter;
12 import java.io.IOException;
13 import java.time.Duration;
14 import java.util.List;
15
16 public class NavigationTrainingResult extends DriverSetup {
17     protected void navigateTo(String menuItem) {
18         WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds
19             (15));
20         Actions actions = new Actions(driver);
21
22         try {
23             WebElement serviceBtn = wait.until(
24                 ExpectedConditions.elementToBeClickable(
25                     By.xpath("//span[text()='Service']")
26                 )
27             );
28             actions.moveToElement(serviceBtn).perform();
29             Thread.sleep(800);
30
31             WebElement targetBtn = wait.until(
32                 ExpectedConditions.elementToBeClickable(
33                     By.xpath("//span[text()='\" + menuItem + \"']")
34                 )
35             );
36
37             ((JavascriptExecutor) driver).executeScript(
38                 "arguments[0].scrollIntoView(true);", targetBtn
39             );
40
41             Thread.sleep(500);
42             targetBtn.click();
43             Thread.sleep(1500);
44
45         } catch (Exception e) {
46             throw new RuntimeException("Ошибка перехода: " + menuItem, e);
47         }
48     }
49
50     private void saveDebugInfo(String testName, Exception e) {
51         try {
52             File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType
53                 .FILE);
54             src.renameTo(new File("error_" + testName + ".png"));
55
56             FileWriter writer = new FileWriter("error_" + testName + ".html
57                 ");
```

```

56         writer.write(driver.getPageSource());
57         writer.close();
58
59     } catch (IOException ex) {
60         ex.printStackTrace();
61     }
62 }
63
64 @Test
65 public void testDifferentElementsPageFullCheck() {
66
67     SoftAssert softAssert = new SoftAssert();
68     String testName = "DifferentElements";
69
70     try {
71         // 1. Переход
72         navigateTo("Different elements");
73
74         // 2. Title
75         String title = driver.getTitle().toLowerCase();
76         softAssert.assertTrue(
77             title.contains("different") && title.contains("element")
78             ,
79             "Неверный заголовок: " + title
80         );
81
82         // 3. Кнопки
83         List<WebElement> buttons =
84             driver.findElements(By.cssSelector("input[type='button'
85             ], button"));
86         softAssert.assertTrue(buttons.size() >= 4,
87             "Меньше 4 кнопок: " + buttons.size());
88
89         // 4. Checkbox
90         List<WebElement> checkboxes =
91             driver.findElements(By.cssSelector("input[type='checkbox'
92             ']'"));
93         softAssert.assertTrue(checkboxes.size() >= 4,
94             "Меньше 4 чекбоксов: " + checkboxes.size());
95
96         // 5. Radio
97         List<WebElement> radios =
98             driver.findElements(By.cssSelector("input[type='radio'
99             ']'"));
100         softAssert.assertTrue(radios.size() >= 4,
101             "Меньше 4 радио: " + radios.size());
102
103         // 6. Dropdown
104         List<WebElement> selects = driver.findElements(By.tagName("
105             select"));
106         softAssert.assertTrue(selects.size() >= 1,
107             "Dropdown не найден");
108
109         if (!selects.isEmpty()) {
110             WebElement dropdown = selects.get(0);
111
112             // 7. Видимость + опции
113             softAssert.assertTrue(dropdown.isDisplayed(),
114                 "Dropdown скрыт");
115         }
116     }
117 }

```

```

111         List<WebElement> options = dropdown.findElements(By.tagName
112             ("option"));
113         softAssert.assertTrue(options.size() >= 2,
114             "Мало опций: " + options.size());
115
116         // 8. Проверка значений
117         boolean hasRed = false;
118         boolean hasGreen = false;
119
120         for (WebElement opt : options) {
121             String text = opt.getText().toLowerCase();
122             if (text.contains("red")) hasRed = true;
123             if (text.contains("green")) hasGreen = true;
124         }
125
126         softAssert.assertTrue(hasRed && hasGreen,
127             "Нет Red или Green");
128     }
129
130     // 9. Labels
131     List<WebElement> labels =
132         driver.findElements(By.className("label-checkbox"));
133     softAssert.assertTrue(labels.size() >= 2,
134         "Мало label-checkbox: " + labels.size());
135
136     } catch (Exception e) {
137         saveDebugInfo(testName, e);
138         softAssert.fail("Ошибка в тесте: " + e.getMessage());
139     }
140
141     softAssert.assertAll();
142 }

```